

Vector Space

In the last lesson, you watched a token move through the layers, its numbers changing at every step. Those layers reshape the token’s starting numbers into a new vector: a fresh row of numbers that captures what it means.

In theory, the model could now look that vector up in its full table of token embeddings and see which one it matches. But it can’t: the transformed numbers are one of a kind, lining up with no token in the table.

THE PROBLEM

So how does AI work out what the token means, when its vector matches nothing on file?

To see the answer, go back to the taste test from Embeddings, where Pepsi looked almost exactly like Coke until a Citrus dimension finally told them apart.

Taste Profile - Coke vs. Pepsi vs. Coffee

TOKEN	TOKEN ID	DIMENSIONS						
		SWEET	BITTER	FIZZ	HEAT	CAFFEINE	DARK	CITRUS
 Coke	24317	9	1	10	2	3	8	1
 Pepsi	38106	9	1	10	2	3	8	10
 Coffee	51820	1	9	0	9	8	10	0

Notice how Coke and Pepsi’s vectors (their rows of numbers) sit much closer to each other than either does to Coffee.

This matters for AI. The model measures closeness with **distance**: how far apart two vectors are when you add up the gap in every dimension. The model has established that Coke is more like Pepsi than like Coffee.

MEANING IS A POSITION

Now back to that one-of-a-kind token. The model can’t look its numbers up, so it does what we just did with the drinks: it reads the token’s **position**. This is the idea of **vector space**: a map where every vector sits somewhere, and similar words share a neighborhood.

Let’s demonstrate with the example from the Transformer lesson. How does AI establish the meaning of the word **IT**?

